

# Lab8\_\_TextClassification

November 22, 2023

## 0.1 Methodology

The methodology has been described step by step as follows to classify gender.

### 0.1.1 Step 1- Install Classifiers and import all the important libraries.

```
[ ]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
plt.style.use('fivethirtyeight')
import warnings
warnings.filterwarnings('ignore')

import nltk

import re

from nltk.stem import PorterStemmer # for stemming

from nltk.stem import WordNetLemmatizer # for lemmatization

from nltk.corpus import stopwords
nltk.download('punkt')

nltk.download('stopwords')
nltk.download('wordnet')
from sklearn.preprocessing import LabelEncoder

from sklearn.feature_extraction.text import CountVectorizer

from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.model_selection import train_test_split

from sklearn.naive_bayes import GaussianNB

#from xgboost import XGBClassifier
```

```
#from lightgbm import LGBMClassifier

from sklearn.metrics import accuracy_score

from sklearn.metrics import confusion_matrix
from sklearn.metrics import accuracy_score

from sklearn.metrics import confusion_matrix
from sklearn.model_selection import GridSearchCV
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
```

Step 2 - Read data from CSV file

Data set consists of 20050 and 26 columns of tweeter (X) data.

```
[ ]: data = pd.read_csv("Datasets/gender-classifier.csv",encoding='latin1')
```

```
[ ]: data
```

## 0.2 Step 3 - Shape of data

```
[ ]: data.shape
```

## 0.3 Step 4 - Dropping columns

We are dropping some columns we do not need. We keep the gender we want to classify, the tweet description and the tweet itself.

```
[ ]: data = pd.concat([data.gender,data.description],axis=1)
```

### 0.3.1 Step 6- Null Values

Check Null values in the dataset.

```
[ ]: data.isnull().sum()
```

### 0.3.2 Step 7- Drop Null Values

Null values are dropped using dropna() function.

```
[ ]: data.dropna(axis=0,inplace=True)
```

### 0.3.3 Step 8- Count the 'gender' column.

Count the variables of the 'gender' column.

```
[ ]: data['gender'].value_counts()
```

### 0.3.4 Step 9- Save only 'male' and 'female' variables.

Save only the variables 'male' and 'female' in the 'gender' column as we are concerned about only these two genders and check their count again.

```
[ ]: filtered_data = data[data['gender'].isin(['male', 'female'])]
```

```
[ ]: filtered_data['gender'].value_counts()
```

```
[ ]: filtered_data
```

### 0.3.5 Step 10- Encoding of categorical variables 'male' and 'female'.

Encode the 'male' and 'female' category as 1 and 0 using the replace() function. Male was encoded as 1 and female

```
[ ]: for gen in filtered_data['gender']:
      if gen=='male':
          filtered_data['gender'].replace({'male':'1'},inplace=True)
      elif gen=='female':
          filtered_data['gender'].replace({'female':'0'},inplace=True)
      filtered_data['gender'].value_counts()
```

### 0.3.6 Step11- Cleaning of the description column

keep only the words containing alphanumeric characters and remove punctuations. Defined a function clean() to remove punctuations from the 'description' column.

```
[ ]: def clean(review):
      descrip = re.sub('[^a-zA-Z]', ' ', review)
      review = review.lower()
      return review

      filtered_data['descrip_Cleaned'] = pd.DataFrame(filtered_data['description'].
      ↪apply(lambda x: clean(x)))
      filtered_data.head()
```

Cleaning the data which includes punctuation removal, number removal, and different signs like '@', '(', ')', '#', and URL with ' '.

```
[ ]: url_regex = r"(https?://)?(www\d?\.)?[a-zA-Z0-9-]+\.[a-z]{2,}+(\S*)?"
      filtered_data['descrip_Cleaned'] = filtered_data['descrip_Cleaned'].
      ↪replace(url_regex, "", regex=True)
```

```
[ ]: filtered_data
```

### 0.3.7 Step 12- Tokenization of ‘description\_Cleaned’ column

Tokenization is the process of breaking text into smaller pieces which we know as tokens. Each word, special character, or number in a sentence can be depicted as a token in NLP. Tokenization is the process of breaking down a piece of code into smaller units called tokens.

Tokenization has been performed using the `word_tokenization()` function, which splits text into individual words. Tokenized words were stored in a list named `descrip_cleaned` and after that, a list comprehension was performed using `is.alpha()` function alphabets were only stored in ‘`descrip_new_alpha`’ list.

```
[ ]: from nltk.tokenize import word_tokenize
      filtered_data['descrip_Cleaned'] = [nltk.word_tokenize(tweet) for tweet in
      ↪filtered_data['descrip_Cleaned']]
      descrip_new=[]
      for each_row in filtered_data['descrip_Cleaned']:
          descrip_new.append([i for i in each_row if i.isalpha()])
      descrip_new_alpha=[]
```

### 0.3.8 Step 13- Stopwords removal from the ‘descrip\_Cleaned’ column.

Stopwords do not add meaning to the sentence, so stop words were removed from the sentence by running a list comprehension and words that do not fall under stopwords were again stored in the list ‘`descrip_new_alpha`’.

```
[ ]: stop_words = set(stopwords.words('english'))

      for each_row in descrip_new:

          descrip_new_alpha.append([i for i in each_row if i not in stop_words])
```

### 0.3.9 Step14- Lemmatization of the text of the ‘descrip\_Cleaned’ column.

Lemmatization is an organized and step-by-step process of obtaining the root form of the word. It makes use of vocabulary and morphological analysis. Lemmatization was done to get to the root of any word and using `WordNetLemmatizer()` class an object was created through which lemmatization was performed and then using `join()` function all words were joined into a sentence and the complete cleaned description was stored in a list named ‘`descrip_Cleaned`’.

```
[ ]: description_new_lemma = []

      lemma = nltk.WordNetLemmatizer()

      for each_row in descrip_new_alpha:

          description_new_lemma.append([lemma.lemmatize(word) for word in each_row])

      filtered_data['descrip_Cleaned'] = description_new_lemma
```

```
filtered_data['descrip_Cleaned'] = [" ".join(desc) for desc in_
↳filtered_data['descrip_Cleaned'].values]
```

```
[ ]: filtered_data
```

### 0.3.10 Step 15- Creating a bag-of-words model (Vectorization)

Vectorization is a methodology in NLP to map words and phrases from vocabulary to a corresponding vector of real numbers which is used to find word predictions, word similarities/semantics. To make documents corpora more relatable for computers they must first be converted into some numerical structure. Few techniques are used to achieve this, is called 'Bag of Words'.

CountVectorizer is the most straightforward one, which counts the number of times a token shows up in the document and uses this value as its weight. Words are needed to be encoded into integers so that they can be fed to the input of any machine learning model. For this purpose, Scikit-learn's CountVectorizer() was used to convert a collection of text documents to a vector of term/token counts and maximum features were fixed to 150.

```
[ ]: # %% creating bag of words model
from sklearn.feature_extraction.text import CountVectorizer # for bag of words
cv = CountVectorizer(max_features = 150)
x = cv.fit_transform(filtered_data['descrip_Cleaned']).toarray()
y = filtered_data.iloc[:,0].values # positive or negative comment
```

```
[ ]: print(x.shape, y.shape)
```

### 0.3.11 Finally - Train Test Split

```
[ ]: # train test split
from sklearn.model_selection import train_test_split

x_train, x_test, y_train, y_test = train_test_split(x,y,test_size = 0.
↳1,random_state = 0)
```

```
[ ]: print(x_train.shape, x_test.shape, y_train.shape, y_test.shape)
```

### 0.3.12 Model Training

```
[ ]: gnbmodel = GaussianNB()
gnbmodel.fit(x_train , y_train)
y_pred = gnbmodel.predict(x_test)
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy: %.2f%%" % (accuracy * 100.0))
```

```
[ ]: confusion_matrix(y_test, y_pred)
```

```
[ ]: print(classification_report(y_test, y_pred))
```

### 0.3.13 Step 28- GridsearchCV() was used to tune hyperparameters for three classifiers.

Sometimes we often need to perform tuning of training parameters for optimal performance. This increases the accuracy and tunes the model to perform better. There are methods in scikit learn to tune those parameters automatically. It is called grid search. For Naive Bayes classifier the parameters which were passed to GridsearchCV() {'var\_smoothing': np.logspace(0,-9, num=100)}

```
[ ]: param_grid_nb = {
      'var_smoothing': np.logspace(0,-9, num=100)
    }
nbModel_grid = GridSearchCV(estimator=gnbmodel, param_grid=param_grid_nb,
    ↪ verbose=1, cv=3, n_jobs=-1)
nbModel_grid.fit(x_train, y_train)

print("Best model parameters:\n", nbModel_grid.best_params_)

#The prediction was done using the best hyperparameter evaluated.

y_pred_hyper = nbModel_grid.predict(x_test)

#Confusion Matrix and Accuracy were determined after Hyperparameter tuning of
    ↪ Naive Bayes Classifier.

print(confusion_matrix(y_test, y_pred_hyper), ": is the confusion matrix")
```

### 0.3.14 Classification report

```
[ ]: accuracy_gnb_hyper = accuracy_score(y_test, y_pred_hyper)
print("Accuracy: %.2f%%" % (accuracy_gnb_hyper * 100.0))
```